Machine Learning Applications

Introduction to Keras

Core Topics

- Tensors & shapes
- Models and Layers
- Compile settings
- Training with fit()
- Evaluate and predict

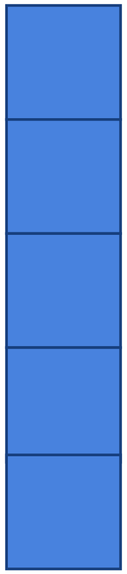
What is Keras?

- TensorFlow is an open-source machine learning and deep learning framework developed by Google.
- Keras is a high-level deep learning API for **building**, **training**, **evaluating**, and **deploying** neural networks. It is integrated directly into TensorFlow.
- It provides:
  - Model
  - Layer
  - Training
  - Prediction

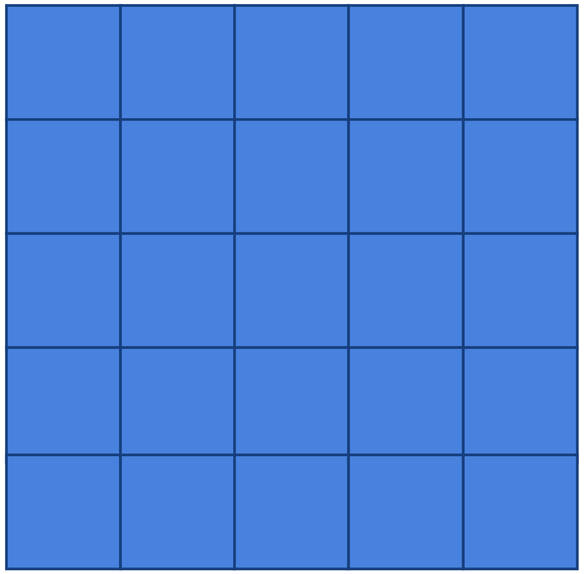
Tensors

- Tensor is a container of numbers with a shape. Keras models expect each sample to match the input shape.
- Tensor is often thought of as generalized matrix
- It is a specialized data structure we used to encode the inputs and outputs of a model, as well as the model's parameters
- You can have 1-D, 2-d, 3-D, n-D tensors

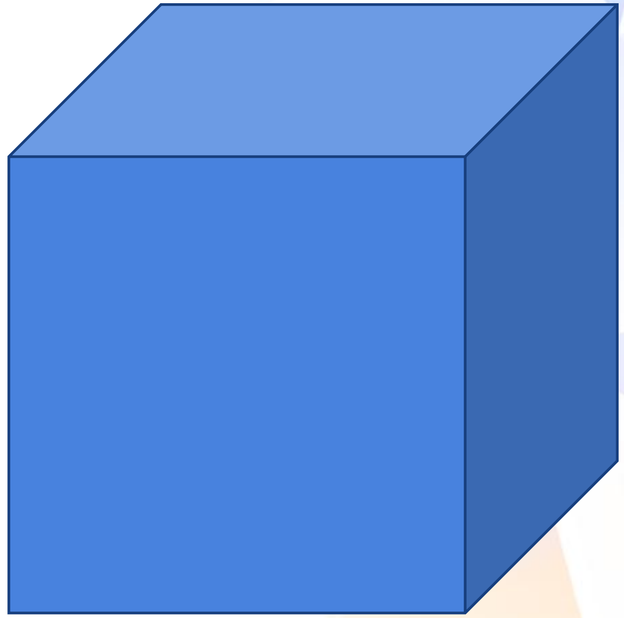
Scalar



Vector 1-D tensor

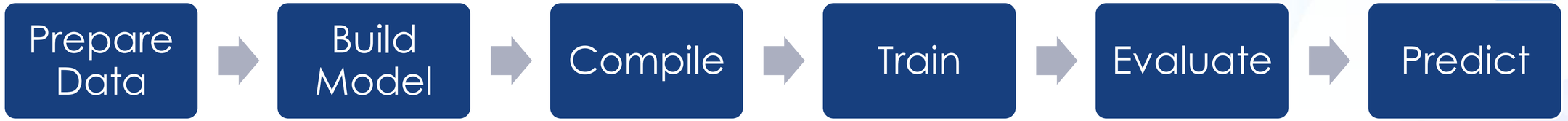


Matrix 2-D tensor



Tensor 3-D tensor

The Basic Keras Workflow



Most Important Keras APIs

1. **Sequential API** -> Simplest architecture, Good for:
   - feedforward NN
   - simple CNNs

2.**Training APIs**. Core methods: compile(), fit(),evaluate(), predict()

3.**Layers API**. The layers API contains all neural network building blocks.

Dense layer, ReLU, softmax, Conv2D, MaxPooling2D, LSTM, Dropout etc.

4. **Optimizer API**. Optimizers update weights during training. Key optimizers: SGD, Adam etc.

5. **Losses API**. Loss functions measure prediction error. MSE, MAE, crossentropy etc.

6. **Metrics API**. Metrics evaluate model performance.

7. **Keras Applications**. Provide pretrained deep learning models. ResNet, VGG16, DenseNet, EfficientNet, DenseNet etc.

Sequential API

- The Keras Sequential API is the simplest way to build a neural network by stacking layers linearly:

  Input->Layer 1-> Layer 2->output Layer

  - It is good for one input tensor and one output tensor.

  Syntax:

```python
# Define Sequential model with 3 layers
model = keras.Sequential(
    [
        layers.Dense(2, activation="relu", name="layer1"),
        layers.Dense(3, activation="relu", name="layer2"),
        layers.Dense(4, name="layer3"),
    ]
)
# Call model on a test input
x = ops.ones((3, 3))
y = model(x)
```

```python
# Create 3 layers
layer1 = layers.Dense(2, activation="relu", name="layer1")
layer2 = layers.Dense(3, activation="relu", name="layer2")
layer3 = layers.Dense(4, name="layer3")

# Call layers on a test input
x = ops.ones((3, 3))
y = layer3(layer2(layer1(x)))
```

Common Sequential Layers

Layers are the basic building blocks of neural networks. A layer takes a tensor as input, perform a computation, and returns another tensor. Some layers also contain trainable weights.

Layer	Description
Dense	Fully connected layer
Conv2D	Convolution layer
MaxPooling2D	Downsampling
Flatten	Convert tensor → vector
Dropout	Reduce overfitting
LSTM	Recurrent layers

Model Training APIs

- Keras model training APIs are the methods you call after building a model to configure, train, evaluate, and use it.
- Main training APIs: `compile`, `fit`, `evaluate`, `predict`, `train_on_batch`, `test_on_batch` and `history`
- Compile method
  - Configure the models for training. It configures which optimizer to use, which loss function to minimize, which metrics to monitor
  - Example code:

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.BinaryCrossentropy(),
    metrics=[
        keras.metrics.BinaryAccuracy(),
        keras.metrics.FalseNegatives(),
    ],
)
```

Fit() method

- Fit method is the main training method. It defines the training datasets, epochs, batch size, validation data, etc.
- `fit()` trains the models for a fixed number of epochs, and return a history object which stores loss and metric values
- `fit()` trains by slicing data into batches and iterating over the dataset for the specified number of epochs.

Model evaluate

- `Model.evaluate()` tests the trained model on data.
- It is used to check performance on unseen data. It does not train or update weights.

```
Model.evaluate(
    x=None,
    y=None,
    batch_size=None,
    verbose="auto",
    sample_weight=None,
    steps=None,
    callbacks=None,
    return_dict=False,
    **kwargs
)
```

Model Predict

- `Model.predict()` generates model outputs.
- `Predict()` generates output predictions in batches and return Numpy arrays of predictions. It is designed for batch prediction.

```
Model.predict(x, batch_size=None, verbose="auto", steps=None, callbacks=None)
```

Model history

- History is the object returned by
- `history=model.fit()`
- This is a python dictionary that stores the training loss and metric values after each epoch. If you use validation data, it also stores validation loss and metrics.
- Example output

```
{
   "loss": [0.95, 0.72, 0.55, 0.43],
   "accuracy": [0.60, 0.72, 0.81, 0.86],
   "val_loss": [0.90, 0.78, 0.70, 0.75],
   "val_accuracy": [0.62, 0.70, 0.76, 0.74]
}
```

Loss

- Probabilistic losses
  - Binarycrossentropy class
  - CategoricalCrossentropy class
  - SparseCategoricalCrossentropy class
- Regression losses
  - MeanSquaredError class
  - MeanAbsoluteError class
  - MeanAbsolutePercentageError class
  - MeanSquaredLogarithmicError class
  - CosineSimilarity class

Common used Metrics

Regression	**mae, mse, rmse**
Binary classification	Accuracy, precision, recall, auc
Multi-class classification	Accuracy
Multi-label classification	Precision, recall, F1

Common Model Architecture

- Dense neural network – tabular data and general vector inputs.
- CNN – image data and spatial pattern extraction.
- RNN/LSTM – sequences, time series
- Antoencoder – representation learning and reconstruction

Epoch, batch size

- Epoch is the model has gone through the entire training dataset one time.
- Batch size is how many samples the model uses before updating its weights once.
- Examples:
  - Number of training samples = 1000
  - batch_size = 100
  - epochs = 5

  So in each epoch:

  Batch 1: 100 samples → update weights
  Batch 2: 100 samples → update weights
  Batch 3: 100 samples → update weights
  …
  Batch 10: 100 samples → update weights

  Weights updates 10 times.